PagedAttention & KV-Cache Memory.

LLM serving is not bottlenecked by FLOPs — it is bottlenecked by the *memory* the KV-cache consumes. Naive serving reserves a contiguous buffer for the maximum possible sequence length of *every* request, so a short request strands most of its reservation as dead space. PagedAttention borrows the operating system's idea of virtual memory: split the cache into fixed-size blocks, store them non-contiguously, and look them up through a per-request block table. Internal waste collapses from 95% to under one block per sequence, and the number of requests that fit in a fixed GPU budget rises several-fold. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$\text{bytes/token} = 2\,L\,d\,b_{\text{bytes}} \qquad \text{waste}_{\text{naive}} = 1 - \frac{T_{\text{used}}}{T_{\max}}$$

Worked example. $L = 32$, $d = 4096$, fp16 (b=2): bytes/token = 524,288 (512 KiB); a 16-token block = 8 MiB. A request using 100 tokens reserves $T_{\max}$=2048 under naive serving (1 GiB, 95.1% wasted) but only 7 blocks (56 MiB) under paging.

Read top to bottom: each step feeds the next.

Contents

1 The claim: memory, not compute, caps the batch

At decode time the model processes one new token per step, but it must *read* every past token's keys and values from the KV-cache. That cache is the binding resource: the more requests you can keep resident, the larger the batch, and decode throughput rises almost linearly with batch size (Topic 3). The cache size per cached token is fixed by the architecture:

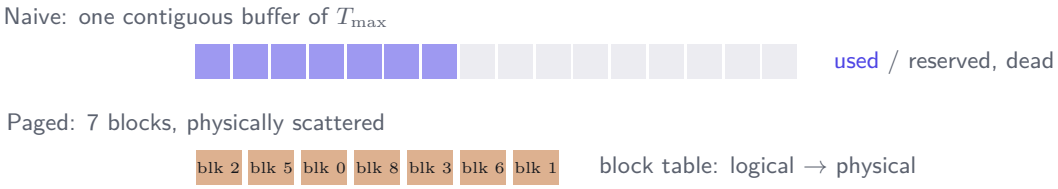
$$\boxed{\text{bytes/token} \;=\; 2\,L\,d\,b_{\text{bytes}}} \tag{1}$$

The leading 2 counts keys *and* values; L is the layer count, d the model width (summed over heads), and b_{bytes} the bytes per scalar. For a token sequence of length T the cache is T times this. The problem is not Eq. (1) — it is *how that cache is allocated*.

Symbol	Meaning	Value / shape
L	transformer layers	32
d	model width ($H\,d_{\text{head}}$)	4096
b_{bytes}	bytes per scalar	2 (fp16)
B_{tok}	tokens per block	16
$T_{\max}$	reserved max length (naive)	2048
T_{used}	tokens actually used	here 100
M	GPU KV budget	24 GiB

A naive server is like booking an entire 2048-seat stadium for every visitor in case they bring friends — then most arrive alone. PagedAttention hands out seats a row of 16 at a time, on demand, from a shared pool. Nobody holds an empty stadium, so far more visitors fit at once. The cache is the same size physically; the win is purely in how it is *parcelled out*.

2 A diagram: contiguous reservation vs paged blocks



Top: the naive buffer reserves $T_{\max}$=2048 slots; only 100 are live (blue), the rest is *internal fragmentation* (grey). Bottom: paging allocates seven 16-token blocks wherever free memory exists; a block table maps logical positions to physical blocks, so the attention kernel reads the right pages despite non-contiguous storage.

3 The derivation: per-token, per-block, and waste

Per token. With $L = 32$, $d = 4096$, $b_{\text{bytes}} = 2$:

$$2 \cdot 32 \cdot 4096 \cdot 2 = 524{,}288 \text{ bytes} = 512 \text{ KiB per token.}$$

Per block. A block of $B_{\text{tok}} = 16$ tokens therefore costs

$$16 \times 524{,}288 = 8{,}388{,}608 \text{ bytes} = 8 \text{ MiB.}$$

Naive reservation. A request reserves $T_{\max} = 2048$ tokens regardless of use:

$$2048 \times 524{,}288 = 1{,}073{,}741{,}824 \text{ bytes} = 1.0000 \text{ GiB per request.}$$

Paged allocation. A 100-token request needs $\lceil 100/16 \rceil = 7$ blocks (112 token slots):

$$7 \times 8 \text{ MiB} = 56 \text{ MiB per request.}$$

Wasted fraction. Naive strands everything beyond the 100 live tokens:

$$\boxed{\text{waste}_{\text{naive}} = 1 - \frac{T_{\text{used}}}{T_{\max}} = 1 - \frac{100}{2048} = 0.9512 = 95.12\%} \tag{2}$$

Paging wastes only the unused tail of the *last* block: $112 - 100 = 12$ slots, i.e. $12/112 = 10.7\%$ of the (tiny) allocation, and *at most one block* per sequence regardless of length. As sequences grow, that single trailing block becomes negligible.

4 Worked example: how many requests fit in 24 GiB

On a 40 GB A100 a 7B fp16 model takes ≈ 14 GB of weights; after activations and workspace, take $M = 24$ GiB as the KV budget. **Naive** pins 1.0 GiB per request:

$$\left\lfloor \frac{24 \text{ GiB}}{1.0 \text{ GiB}} \right\rfloor = 24 \text{ concurrent requests.}$$

Paged, with a realistic average occupancy of 256 tokens ($\lceil 256/16 \rceil = 16$ blocks $= 0.125$ GiB) allocated on demand:

$$\left\lfloor \frac{24 \text{ GiB}}{0.125 \text{ GiB}} \right\rfloor = 192 \text{ concurrent requests} \quad \Rightarrow \quad \frac{192}{24} = 8\times \text{ more.}$$

The full comparison, sweeping the typical real occupancy:

Avg tokens used	blocks	paged GiB/req	naive fits	paged fits
100	7	0.0547	24	438
256	16	0.1250	24	192
512	32	0.2500	24	96
1024	64	0.5000	24	48
2048	128	1.0000	24	24

Naive always fits exactly 24 (it reserves $T_{\max}$ no matter the real length). Paging fits more whenever the real length is below $T_{\max}$ — which is almost always. The heatmap shades the paged fit count (intensity $\propto$ requests, capped at `indigo!80`):

	100	256	512	1024	2048
paged fits	**438**	192	96	48	24

(Shading is linear in fit count relative to the largest, 438.) In practice request lengths are mixed, and continuous batching (Topic 3) adds its own overheads, so measured *throughput* gains land around 2–4× over naive contiguous serving even though raw capacity can rise 8× or more for short workloads.

> The KV-cache size is fixed by the architecture (Eq. 1); PagedAttention changes nothing about it. The entire gain comes from *eliminating reserved-but-unused memory*. By allocating in small blocks on demand, internal fragmentation drops from ∼95% to under one block per sequence, and that freed memory converts directly into a larger batch and higher throughput.

5 Block sharing: copy-on-write prefixes

Because logical positions are decoupled from physical blocks, two sequences can *point the block table at the same physical block*. A shared system prompt, or the common prefix in beam search, is stored once and referenced by many sequences. When a sequence first diverges (writes into a shared block), the server copies just that one block — *copy-on-write* — and updates only the divergent table entry. For k beams sharing a P-token prefix this turns $k\,P$ cached tokens into $\approx P$, a near-$k\times$ saving on the prefix with no quality change.

6 Properties / consequences that matter

1. **Near-zero internal waste.** Waste is at most one partially-filled block per sequence, independent of $T_{\max}$ — the ≤ 1 block guarantee.
2. **No external fragmentation.** All blocks are the same size, so any freed block fits any new request; memory never becomes unusable through irregular holes.
3. **Batch scales with freed memory.** Reclaimed reservation becomes batch capacity, and decode throughput is roughly linear in batch size up to the bandwidth knee (Topic 3).
4. **Sharing for free.** Indirection through the block table makes prefix sharing and copy-on-write a table operation, not a data copy.

The block size is a genuine trade-off, not a free parameter. **Too large** (e.g. 256 tokens) and internal fragmentation returns — a 100-token request still wastes most of a 256-slot block, recreating the very problem paging solves. **Too small** (e.g. 1 token) and the block table and per-block kernel bookkeeping balloon: more pointer chasing, more metadata, worse memory locality in the attention kernel. Production systems land around 16–32 tokens per block as the sweet spot.

7 Where this sits in the track

Module 0.2 derived the KV-cache and its linear-in-T growth; Module 3.1 (GQA) shrank the cache by cutting KV heads. This topic attacks the *allocation* of whatever cache remains: even a perfectly sized cache is wasted if reserved contiguously. It pairs directly with Topic 3 (continuous batching), which consumes the freed memory as larger in-flight batches, and with Topic 2 (quantization), which shrinks each cached scalar so still more sequences fit.

A Reproducible (NumPy)

Inputs: $L = 32$, $d = 4096$, $b_{\text{bytes}} = 2$, block $= 16$ tokens, $T_{\max} = 2048$, KV budget 24 GiB. The snippet reproduces bytes/token (524,288), bytes/block (8 MiB), the 95.12% naive waste, and the naive-vs-paged fit counts.

```
import numpy as np
L, d, b, block = 32, 4096, 2, 16
per_token = 2*L*d*b                        # 524288 bytes  (512 KiB)
per_block = per_token*block                # 8388608 bytes (8 MiB)
print(per_token, per_block)

Tmax, used = 2048, 100
naive = Tmax*per_token                      # 1073741824 = 1.0 GiB
blocks = int(np.ceil(used/block))           # 7
paged = blocks*per_block                    # 58720256   = 56 MiB
print("waste_naive = %.4f"%(1-used/Tmax)) # 0.9512

M = 24*2**30                                # 24 GiB KV budget
for avg in [100,256,512,1024,2048]:
```

```
    blk = int(np.ceil(avg/block))
    print(avg, blk, "naive=%d"%(M//naive), "paged=%d"%(M//(blk*per_block)))
# 100  7   naive=24 paged=438
# 256  16  naive=24 paged=192
# 2048 128 naive=24 paged=24
```